

Senior Developer Challenge

AI Agent ile Task'tan Pull Request Oluşturma

Merhaba,

Bu teknik challenge'da senden, bir task management sisteminde açılan geliştirme talebini okuyarak ilgili repository üzerinde kod değişikliği yapan ve Pull Request açan bir **AI Development Agent** geliştirmeni bekliyoruz.

Bu case'in amacı yalnızca klasik backend geliştirme yetkinliğini değil; aynı zamanda AI agent kullanımı, repository analizi, kod üretimi, test çalıştırma, Git operasyonları ve production-ready düşünme becerilerini de değerlendirmektir.

1. Senaryo

Şirket içinde ekipler Jira, Trello, GitHub Issues veya benzeri task management sistemleri üzerinden geliştirme talepleri açmaktadır.

Bir task içerisinde aşağıdaki bilgiler yer alır:

- Hangi repository üzerinde çalışılacağı
- Hangi branch'in baz alınacağı
- Hangi feature'ın veya değişikliğin yapılacağı
- Acceptance criteria
- Beklenen test davranışı

Senden beklenen, bu task bilgilerini alan bir sistem geliştirmen. Sistem, task içeriğini analiz etmeli, ilgili repository'yi clone etmeli, AI kullanarak gerekli kod değişikliğini yapmalı, testleri çalıştırmalı ve sonunda bir Pull Request oluşturmalıdır.

2. Örnek Task

Aşağıdaki task örnek olarak verilmiştir. Challenge sırasında bu formatı baz alabilirsin.

```
{
  "taskId": "TASK-123",
  "title": "Add email validation to user registration API",
  "description": "Repository: https://github.com/example-company/user-service\nBranch: develop\n\nRequirement:\nUser registration endpoint currently accepts invalid email formats. Add email format validation to the POST /users/register endpoint.\n\nAcceptance Criteria:\n- If email format is invalid, API should return HTTP 400\n- Error message should be: Invalid
```

```
email format\n- Existing valid registration flow should continue working\n- Add or update unit tests"\n}
```

İstersen kendi oluşturacağın public/private test repository'si üzerinde de çalışabilirsin. Ancak demo sırasında uçtan uca çalışan bir akış görmeyi bekliyoruz.

3. Beklenen Akış

Geliştireceğin sistem aşağıdaki akışı desteklemelidir:

```
Task Input
  ↓
Task Parsing
  ↓
Repository Clone
  ↓
Repository Analysis
  ↓
AI ile Kod Değişikliği
  ↓
Test Çalıştırma
  ↓
Branch Oluşturma
  ↓
Commit
  ↓
Pull Request Oluşturma
  ↓
Execution Report
```

4. Fonksiyonel Gereksinimler

4.1 Task Input Alma

Sistem task bilgisini aşağıdaki yöntemlerden en az biriyle almalıdır:

- REST API endpoint
- CLI komutu
- GitHub Issue
- Jira webhook simülasyonu
- Trello webhook simülasyonu

Minimum kabul edilen örnek:

```
POST /api/tasks
Content-Type: application/json
{
  "taskId": "TASK-123",
  "title": "Add email validation to user registration API",
```

```
"description": "Repository: https://github.com/example-company/user-service\nBranch: develop\n\nRequirement:\nAdd email validation to POST /users/register endpoint.\n\nAcceptance Criteria:\n- Invalid email returns HTTP 400\n- Error message should be Invalid email format\n- Add or update unit tests"
```

4.2 Task Parsing

Sistem task içeriğinden aşağıdaki bilgileri çıkarmalıdır:

```
{
  "taskId": "TASK-123",
  "repositoryUrl": "https://github.com/example-company/user-service",
  "baseBranch": "develop",
  "requirement": "Add email validation to POST /users/register endpoint",
  "acceptanceCriteria": [
    "Invalid email returns HTTP 400",
    "Error message should be Invalid email format",
    "Add or update unit tests"
  ]
}
```

Parsing işlemini rule-based, AI-based veya hybrid şekilde yapabilirsin.

4.3 Repository Clone

Sistem task içerisindeki repository'yi clone etmelidir.

Beklentiler:

- Repository URL validate edilmeli
 - Base branch checkout edilmeli
 - Her task için izole bir workspace kullanılmalı
 - Private repo kullanılacaksa token desteği sağlanmalı
 - Aynı task tekrar çalıştırıldığında sistemin nasıl davranacağı açıklanmalı
-

4.4 Repository Analysis

Sistem repository üzerinde analiz yapmalıdır.

Beklenen analiz çıktıları:

- Proje dili
- Framework
- Build tool
- Test komutu

- Muhtemel ilgili dosyalar
- Değişiklik yapılacak alanlar
- Varsa mevcut test dosyaları

Örnek çıktı:

```
{
  "language": "Java",
  "framework": "Spring Boot",
  "buildTool": "Maven",
  "testCommand": "mvn test",
  "relevantFiles": [
    "src/main/java/com/example/user/UserController.java",
    "src/main/java/com/example/user/UserService.java",
    "src/test/java/com/example/user/UserControllerTest.java"
  ]
}
```

4.5 AI ile Kod Değişikliği

Bu challenge’da AI kullanımı zorunludur.

AI kısmı mock olmamalıdır. Gerçek bir AI modeli kullanılmalıdır.

Dilediğin AI modelini veya sağlayıcıyı kullanabilirsin:

- OpenAI
- Anthropic
- Google Gemini
- GitHub Copilot tabanlı akışlar
- Local LLM
- Başka bir AI modeli

Beklentiler:

- AI, task gereksinimini ve repository context’ini kullanarak kod değişikliği yapmalıdır.
- AI’ın hangi dosyaları değiştirdiği raporlanmalıdır.
- Kod değişikliği sadece ilgili alanlarla sınırlı olmalıdır.
- Mevcut davranışlar gereksiz yere bozulmamalıdır.
- Acceptance criteria karşılanmalıdır.
- Test eklenmeli veya mevcut testler güncellenmelidir.

AI kullanımında aşağıdaki konulara özellikle dikkat etmeni bekliyoruz:

- Repository context’i modele nasıl veriliyor?
- Çok büyük repository’lerde context nasıl sınırlandırılıyor?
- Modelin yanlış dosya değiştirmesi nasıl engelleniyor?
- AI çıktısı nasıl doğrulanıyor?
- Secret, token veya hassas kodların modele gönderilmesi nasıl kontrol ediliyor?

- Prompt injection riskleri nasıl ele alınıyor?

4.6 Test Çalıştırma

Kod değişikliği sonrasında sistem testleri çalıştırılmalıdır.

Beklentiler:

- Test komutu otomatik tespit edilmeli veya config üzerinden alınmalı
- Test sonucu execution report'a eklenmeli
- Test başarısız olursa sistemin davranışı net olmalı

Örnek test sonucu:

```
{
  "testStatus": "passed",
  "command": "mvn test",
  "duration": "12s"
}
```

Test başarısız olursa aşağıdaki yaklaşımlardan birini seçebilirsiniz:

- PR açma ve hatayı raporla
- PR aç ama açıklamada testlerin başarısız olduğunu belirt
- AI'a test sonucunu vererek düzeltme denemesi yap
- Belirli sayıda retry mekanizması uygula

Seçtiğin yaklaşımı README'de açıklaman beklenir.

4.7 Branch, Commit ve Pull Request

Sistem yeni bir branch oluşturmali, değişiklikleri commit etmeli ve Pull Request açmalıdır.

Branch adı örneği:

```
ai-agent/TASK-123-email-validation
```

Commit mesajı örneği:

```
TASK-123 Add email validation to user registration
```

PR başlığı örneği:

```
TASK-123 Add email validation to user registration API
```

PR açıklaması aşağıdaki bilgileri içermelidir:

```
## Summary
Added email validation to the user registration endpoint.
```

```
## Task
TASK-123
```

```
## Changes
- Added email format validation
- Added HTTP 400 response for invalid email
- Added or updated unit tests
```

```
## Test Result
Command: mvn test
Status: Passed
```

```
## AI Usage
Model: <used-model-name>
Changed files:
- src/main/java/...
- src/test/java/...
```

```
## Generated By
AI Development Agent
```

Minimum beklenti GitHub üzerinde PR açılmasıdır. GitLab veya Bitbucket kullanmak istersen bu da kabul edilebilir.

5. Teknik Seçimler

Teknoloji seçimi sana bırakılmıştır.

Backend için aşağıdakilerden birini kullanabilirsin:

- Go
- Java / Spring Boot
- Python / FastAPI
- Node.js / NestJS
- Başka bir backend teknolojisi

Git provider olarak aşağıdakilerden birini kullanabilirsin:

- GitHub
- GitLab
- Bitbucket

AI modeli olarak dilediğin gerçek modeli kullanabilirsin. Mock AI kabul edilmeyecektir.

6. Non-Functional Beklentiler

6.1 Güvenlik

Aşağıdaki konular değerlendirme kapsamında olacaktır:

- Token ve secret yönetimi
 - Repository allowlist / denylist yaklaşımı
 - Private repository güvenliği
 - Workspace izolasyonu
 - Command injection riskleri
 - Prompt injection riskleri
 - AI modeline gönderilen context'in sınırlandırılması
 - Hassas verilerin modele gönderilmesini engelleme
 - Task açıklaması içinden gelebilecek zararlı yönlendirmelere karşı önlem
-

6.2 Observability

Sistem çalışırken anlamlı log üretmelidir.

Beklenen log adımları:

- Task alındı
- Task parse edildi
- Repository clone başladı
- Repository clone tamamlandı
- Repository analizi tamamlandı
- AI kod değişikliği yaptı
- Testler çalıştırıldı
- Branch oluşturuldu
- Commit atıldı
- PR açıldı
- Hata oluştuysa detaylı hata loglandı

Bonus:

- Execution timeline
 - JSON report
 - Trace ID
 - AI token usage / cost bilgisi
 - Değiştirilen dosyaların listesi
-

6.3 Hata Yönetimi

Aşağıdaki senaryoları ele alman beklenir:

- Repository clone edilemedi
 - Branch bulunamadı
 - Task parse edilemedi
 - AI yeterli değişikliği yapamadı
 - AI hatalı veya alakasız dosya değiştirdi
 - Testler başarısız oldu
 - Git push başarısız oldu
 - PR oluşturulamadı
 - Aynı task için daha önce PR açılmış
 - Token veya permission hatası oluştu
-

7. Teslim Edilecekler

Challenge sonunda aşağıdaki çıktıları bekliyoruz:

7.1 Source Code

Çalışan uygulamanın kaynak kodu.

7.2 README

README içerisinde aşağıdaki bilgiler olmalıdır:

- Projenin amacı
 - Kullanılan teknoloji stack'i
 - Kullanılan AI modeli
 - Kurulum adımları
 - Environment variable'lar
 - Nasıl çalıştırılır?
 - Örnek task payload
 - Örnek execution report
 - Mimari açıklama
 - AI agent akışı
 - Güvenlik yaklaşımı
 - Bilinen kısıtlar
 - Test başarısız olduğunda sistem davranışı
 - Production'a almak için yapılması gereken iyileştirmeler
-

7.3 Demo

Aşağıdakilerden en az biri beklenmektedir:

- Çalışan local demo
- Video demo
- Terminal çıktısı
- Oluşturulmuş örnek Pull Request linki
- Execution report çıktısı

Demo sırasında aşağıdaki akışı görmek istiyoruz:

Task verildi → Repo clone edildi → AI değişiklik yaptı → Test çalıştı → PR açıldı

7.4 Architecture Diagram

Basit bir mimari diyagram beklenmektedir.

Örnek:

```
Task Management System
  |
  v
Task API / Webhook
  |
  v
Task Parser
  |
  v
Repository Manager
  |
  v
Repository Analyzer
  |
  v
AI Code Agent
  |
  v
Test Runner
  |
  v
Git Provider Client
  |
  v
Pull Request
```

8. Bonus Özellikler

Aşağıdaki özellikler zorunlu değildir, ancak artı puan sağlar:

- Jira webhook entegrasyonu
- Trello webhook entegrasyonu
- GitHub Issue üzerinden tetikleme
- Dry-run mode

- Human approval step
 - PR açmadan önce diff preview
 - Docker sandbox içinde test çalıştırma
 - Multi-repo support
 - AI retry mechanism
 - AI'a test failure output vererek otomatik düzeltme
 - LLM token usage / cost tracking
 - Agent execution report
 - Repository context indexing
 - Büyük repository'ler için smart context selection
 - Multiple agent yaklaşımı:
 - Task Parser Agent
 - Repo Analyzer Agent
 - Code Writer Agent
 - Test Fixer Agent
 - PR Writer Agent
-

9. Değerlendirme Kriterleri

Fonksiyonel Başarı

- Task input alınabiliyor mu?
 - Task doğru parse ediliyor mu?
 - Repository clone ediliyor mu?
 - AI gerçekten kullanılıyor mu?
 - Kod değişikliği doğru yapılıyor mu?
 - Testler çalıştırılıyor mu?
 - Branch, commit ve PR akışı çalışıyor mu?
-

AI Kullanımı

- Gerçek AI modeli kullanılmış mı?
 - Repository context'i modele anlamlı şekilde verilmiş mi?
 - AI çıktısı doğrulanmış mı?
 - Değişiklikler acceptance criteria ile uyumlu mu?
 - AI'ın yanlış veya fazla değişiklik yapması engellenmiş mi?
 - Prompt tasarımı anlaşılır mı?
 - Token / context yönetimi düşünülmüş mü?
-

Mimari Kalite

- Sistem modüler mi?
 - AI katmanı soyutlanmış mı?
 - Git provider katmanı soyutlanmış mı?
 - Test runner ayrı bir component olarak düşünülmüş mü?
 - Hata yönetimi yeterli mi?
 - Config yönetimi düzgün mü?
-

Güvenlik

- Token'lar güvenli yönetiliyor mu?
 - Repository erişimi kontrollü mü?
 - Workspace izolasyonu var mı?
 - Command injection riski azaltılmış mı?
 - Prompt injection riski düşünülmüş mü?
 - Private code'un AI modeline gönderilmesiyle ilgili riskler değerlendirilmiş mi?
-

Kod Kalitesi

- Kod okunabilir mi?
 - Clean code prensiplerine uygun mu?
 - Test yazılmış mı?
 - README yeterli mi?
 - Hata mesajları anlaşılır mı?
 - Genişletilebilir bir yapı var mı?
-

Sunum ve Açıklama

- Aday çözümünü net anlatabiliyor mu?
 - Trade-off'ları açıklayabiliyor mu?
 - Neleri bilerek scope dışında bıraktığını söyleyebiliyor mu?
 - Production'a geçiş için neler yapılması gerektiğini ifade edebiliyor mu?
-

10. Kısıtlar

- AI kullanımı gerçek olmalıdır; mock AI kabul edilmez.
- Token, secret veya credential repository'ye commit edilmemelidir.
- Kod değişiklikleri mümkün olduğunca ilgili requirement ile sınırlı kalmalıdır.
- PR insan review'undan geçecek şekilde hazırlanmalıdır.
- Test sonucu PR açıklamasında veya execution report'ta gösterilmelidir.

- Sistem, başarısız adımları sessizce geçmemelidir.
- README olmadan teslim eksik kabul edilir.

11. Süre

Bu challenge için önerilen süre:

5 gün

Teslim sonrası kısa bir teknik görüşme yapılacaktır. Bu görüşmede çözümünü, mimari tercihlerini, AI kullanım şeklini, güvenlik yaklaşımını ve production'a alma stratejini anlatmanı bekliyoruz.

12. Kısa Özet

Bu challenge'da senden beklenen şey:

```
Bir task input'u al.  
Task içinden repo, branch ve requirement bilgisini çıkar.  
Repo'yu clone et.  
AI kullanarak gerekli kod değişikliğini yap.  
Testleri çalıştır.  
Yeni branch oluştur.  
Commit at.  
Pull Request aç.  
Sonucu raporla.
```

Teşekkürler